

find path

Find_path

Run command: path_of_exe

- inputFileName: The name of the text
- StartStation : The name of the starting station
- EndStation : The name of the goal station

The network graph structure:

Class Station represents the vertices of our graph.

Struct Connection represents the edges of our class.

In the main function an unordered_map map is used with the name of the station being the key and the value being a pointer station.

The unordered_map is used to keep track of the station that already exist.

.Struct Connection:

fields:

```
Station* stationA;
Station* stationB;
int weight;
std::string lineName;
- Name of the connection (for example U1,U2,U3...)
```

.Class Station:

private:

```
std::string name;
- Name of the station
```

```
std::vector<Connection*> connections;
- List of connection pointers
```

public:

```
Station(const std::string& stationName);
- Constructor that initializes a station
```

```
void addConnection(Connection* conn);
- Method that append a new connection to the connection list of that station
```

```

    void printLines() const;
- Method that prints the names of all the connection in list

void printConnectedStations() const;
- Method that prints the names of all the neighbouring stations

std::string getName() const;
- Getter method that returns the station name

const std::vector<Connection*>& getConnections() const;
- Getter method that returns the list of connections

```

.find_path.cpp:

```

void addTransitEdge(unordered_map<std::string, Station*>& stationMap,
    const std::string& nameA,
    const std::string& nameB,
    int weight,
    const std::string& lineName
);

```

- Accepts the unordered map, 2 station names, name and weight of the connection. This function firstly checks if the map contains station A. If the station doesn't exist in the map then a new station is created (same steps are used for station B). Then a connection is created and is filled with name, weight and the station pointer. At last the connection is added to the connections list of station A and B

```

bool loadNetworkFromFile(const std::string& filename, unordered_map<std::string,
Station*>& stationMap);

```

- Accepts filename and the unordered map. This function reads each line of the file. For each line stations are created using the previous method until the end of the line is reached. This repeats until no more lines are left to read.

```

void freeMemory(unordered_map<std::string, Station*>& stationMap);

```

- Accepts the unordered map. This function is used to free the memory allocated at the end of the code. This function goes through the unordered map and stores all the connections that it finds in a list. Then all the connections in the list are freed and then all the stations are freed

PathResult

```
struct PathResult {  
    std::unordered_map<std::string, int> distances;  
    std::unordered_map<std::string, Station*> previous;  
    bool pathFound;  
};
```

- distances: saves distance to each node
- previous: saves pointer to the previous station
- pathFound, true if a path was found

dijkstra

Finds shortest path to all possible nodes.

Complexity:

$$O((V + E) \cdot \log V)$$

- V = Station count
- E = Edge count

Parameters:

- StationMap graph
- string startName
- string targetName
- If the start and the target nodes can be found in the graph and neither one is a nullptr, save start and target node.
- Initialize PathResult structure with infinite distances and nullptr for all StationMap Elements. Save start node with distance 0.
- Declare Priority Queue with weight and Station* Pairs and add start as the first element. The Priority Queue processes the node with the smallest current distance first
- Declare unordered Map with Name bool values to track the visited stations, to prevent multiple visits to the same node.
- Next the station and the distance with the smallest distance are saved into variables and popped off the queue.
- If the current station was already visited, continue, else mark it as visited.
- Next we check if the target was reached, if true we early exit and set pathFound to true.
- If hasn't been found we check each connected neighbor.
 - Skip the neighbor if it has been visited.

- Calculate the new potential distance.
- if the new path is shorter than the previously known path, the distance is updated with the newDistance and the current station is set as previous in PathResult.

findPath

- Calls dijkstra to find shortest path to all possible nodes and returns PathResult Structure which includes:
 - pathFound: boolean
 - distances: map of shortest distance to all stations
 - previous: map of previous nodes
- If pathFound is false, error output.
- reconstructPath is called to get the correct sequence of stations.
- Retrieves total distance and prints the resulting path.

reconstructPath

Retraces shortest path according to dijkstra. (Backtracking) The Path is then saved in correct sequence as a string vector.

Complexity

$O(n)$

- Initialize empty path vector and check if a path was found in the ResultPath Structure. If foundPath equals false then return empty path.
- Backtracing from the target over the previous Station in path.
- Loop while we haven't reached the start.
 - add current station from previous map, and save the previous station to the current station.
 - check for nullptr in case of some error. Next save the current Station name.
- As the last station we save the start Station, since there is no previous pointer to the start position.
- Next the list is reversed and returned.